

HTTP Analysis Using Wireshark - Embedded Image Traffic

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

10 September 2026

LAB OVERVIEW

This lab focused on capturing browser-generated HTTP traffic for a webpage containing an embedded image, then analyzing how the image was broken into multiple TCP segments and reassembled by Wireshark. I compared the summed segment sizes against the HTTP Content-Length, then extracted the image directly from the packet capture using Export Objects, and verified its integrity by matching SHA-256 hashes against the original file.

OBJECTIVES

- Explain why one webpage can generate multiple HTTP requests and responses.
- Capture a browser request for HTML and an embedded image.
- Identify TCP segments that form one reassembled HTTP response.
- Calculate and explain IP length, TCP payload length and HTTP object size relationships.
- Extract an image using Wireshark Export Objects and tshark.
- Hash the original and extracted image and assess integrity.
- Document limitations caused by cache, HTTPS or incomplete capture.

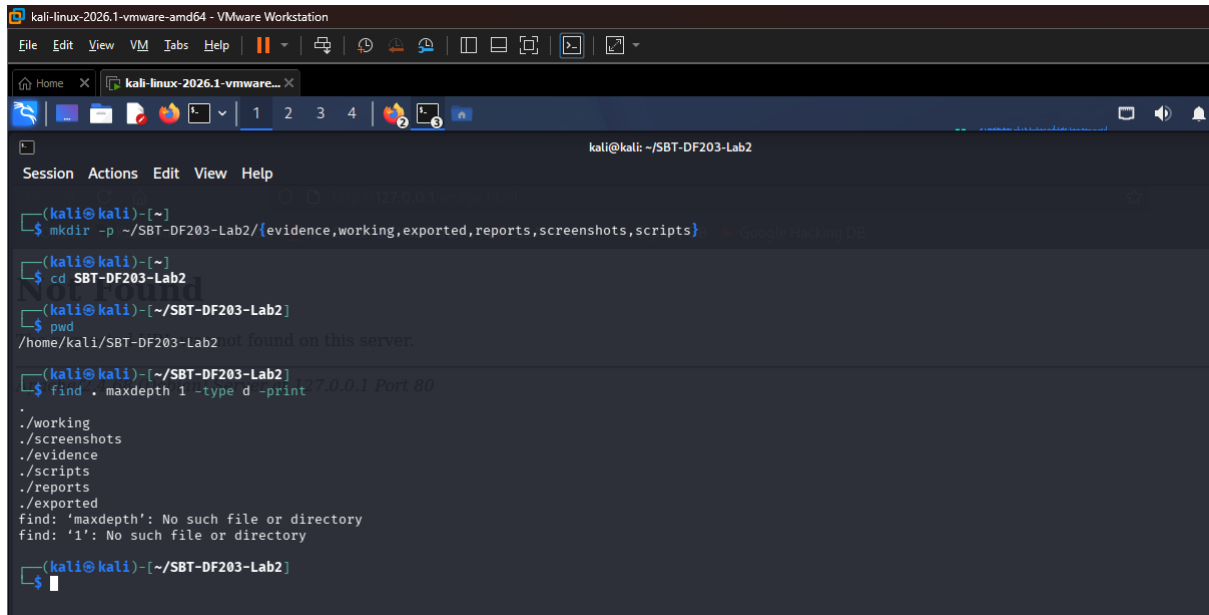
TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Github, Git
- Md5sum and sha256sum
- Apache2
- Firefox, Chrome or Edge
- Wireshark, tshark, Curl, Web browser, file

METHODOLOGY

Lab Folder Structure and Evidence Preparation

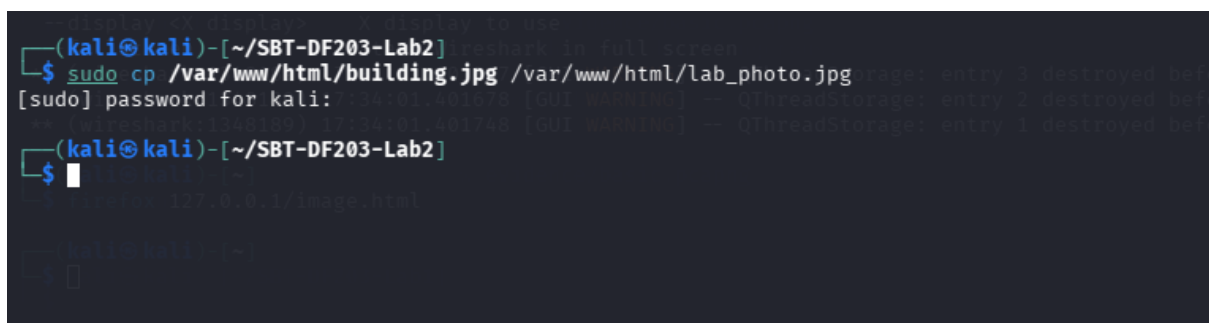
The first step is to create a working environment, I created the folder structure before downloading or generating evidence of any form.



```
kali@kali: ~/SBT-DF203-Lab2
$ mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
$ cd SBT-DF203-Lab2
$ pwd
/home/kali/SBT-DF203-Lab2
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported
find: 'maxdepth': No such file or directory
find: '1': No such file or directory
```

Fig 1: Lab Environment Created

Then we copy the image file to be used into the **/var/www/html** folder and rename it as **lab_photo.jpg**



```
kali@kali: ~/SBT-DF203-Lab2
$ sudo cp /var/www/html/building.jpg /var/www/html/lab_photo.jpg
[sudo] password for kali:
$
```

Fig 2: Copying Image File

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: OJE DOMINION MAYOWA</p></body></html>' | sudo tee /var/www/html/image.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: OJE DOMINION MAYOWA</p></body></html>

(kali@kali)-[~/SBT-DF203-Lab2]
$ firefox 127.0.0.1/image.html

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Fig 3: Editing the Web File, **image.html**

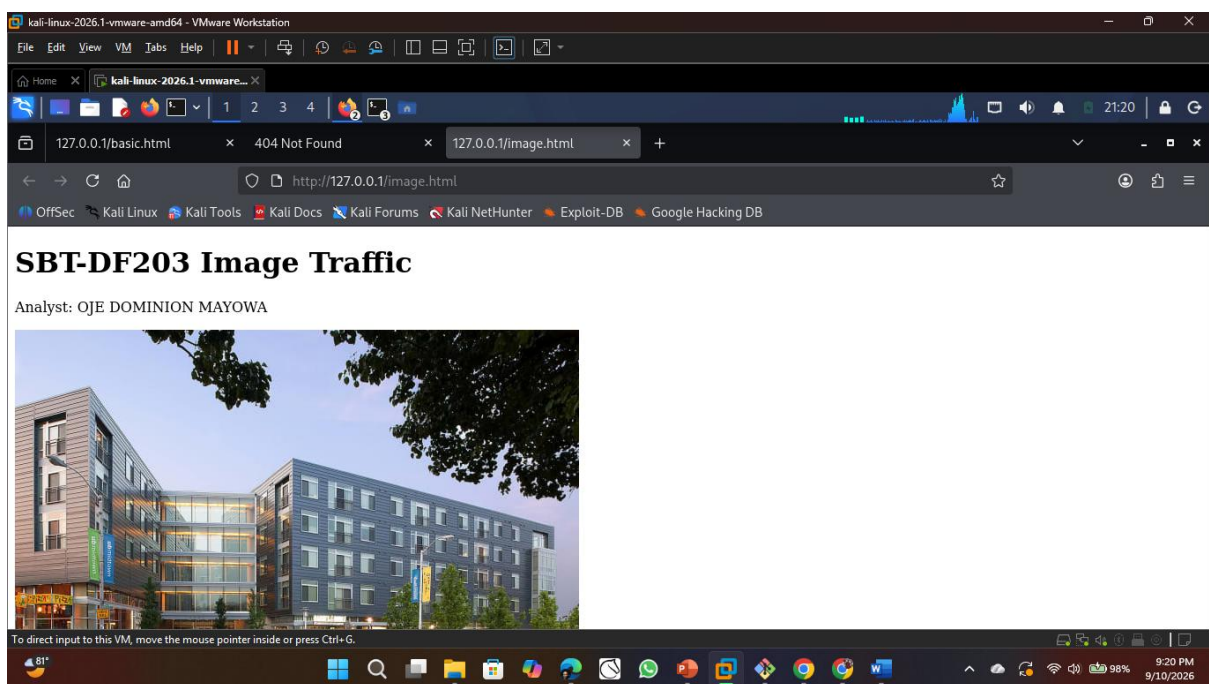


Fig 4: The Web Browser display

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
45dcc75befafda7d7e8fad1ec1677c87cca3354f427c8232a0e3ff18d2fef48 /var/www/html/image.html
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb /var/www/html/lab_photo.jpg

(kali@kali)-[~/SBT-DF203-Lab2]
$
```

Fig 5: Hash Verification of both files

The fig 5 above gave us the sha 256 hash value of both the lab_photo.jpg and the **image.html** file, which shows they are different.

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab2-Oje Dominion Mayowa
Trainee name	OJE DOMINION MAYOWA
Date and time started	10/09/2026, 5:00PM WAT
Evidence file name(s)	lab_photo.jpg, image_traffic.pcapng
Source or generation method	Image downloaded via wget from Dropbox and embedded in basic.html duplicated the image within /var/www/html/ as lab_photo.jpg
Original SHA-256	<pre>(kali@kali)-[~/SBT-DF203-Lab2] \$ cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng (kali@kali)-[~/SBT-DF203-Lab2] \$ sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng tee reports/capture_hashes.txt 3f4c5db511e1c5d4b48e116815d5a1f150d9f40aacfc5160c6ab176c25eab46a evidence/image_traffic.pcapng 3f4c5db511e1c5d4b48e116815d5a1f150d9f40aacfc5160c6ab176c25eab46a working/image_traffic_working.pcapng (kali@kali)-[~/SBT-DF203-Lab2] \$</pre>
Working-copy SHA-256	<pre>(kali@kali)-[~/SBT-DF203-Lab2] \$ cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng (kali@kali)-[~/SBT-DF203-Lab2] \$ sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng tee reports/capture_hashes.txt 3f4c5db511e1c5d4b48e116815d5a1f150d9f40aacfc5160c6ab176c25eab46a evidence/image_traffic.pcapng 3f4c5db511e1c5d4b48e116815d5a1f150d9f40aacfc5160c6ab176c25eab46a working/image_traffic_working.pcapng (kali@kali)-[~/SBT-DF203-Lab2] \$</pre>
Analysis workstation/VM	Kali Linux
Notes on any changes	No alterations were made to the original evidence files. building.jpg was duplicated as lab_photo.jpg purely to create a working copy for embedding and analysis; both remained byte-for-byte identical to the original (confirmed via matching SHA-256 hashes). The image extracted from the packet capture (exported/http_objects/building.jpg) also matched the original file's hash exactly, confirming no data loss or corruption occurred during transmission, capture, or export. Two separate capture files were generated during this lab (image_traffic.pcapng for browser-based traffic and curl_only.pcapng for curl-only traffic) to compare client behavior; neither original capture was modified after creation, only copied into working/ for analysis.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo cp /var/www/html/building.jpg evidence/building.jpg
[sudo] password for kali:
(kali@kali)-[~/SBT-DF203-Lab2]
$ ls
evidence  exported  reports  screenshots  scripts  working
(kali@kali)-[~/SBT-DF203-Lab2]
$ sudo cp /var/www/html/lab_photo.jpg working/lab_photo_working.jpg
(kali@kali)-[~/SBT-DF203-Lab2]
$
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 6: Preserving the evidence

Part A - Verify the Webpage and Eliminate Cache Effects

First, we check the file type and the content of the file using the **file** command and the **identify** command.

```
(kali@kali)-[~/SBT-DF203-Lab2]
$ file /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 640x415, components 3
(kali@kali)-[~/SBT-DF203-Lab2]
$ identify /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg JPEG 640x415 640x415+0+0 8-bit sRGB 68661B 0.000u 0:00.000
(kali@kali)-[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 156 Sep 10 21:19 /var/www/html/image.html
-rw-r--r-- 1 root root 68K Sep 10 21:15 /var/www/html/lab_photo.jpg
(kali@kali)-[~/SBT-DF203-Lab2]
$
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 7: Identifying the File type, File Size

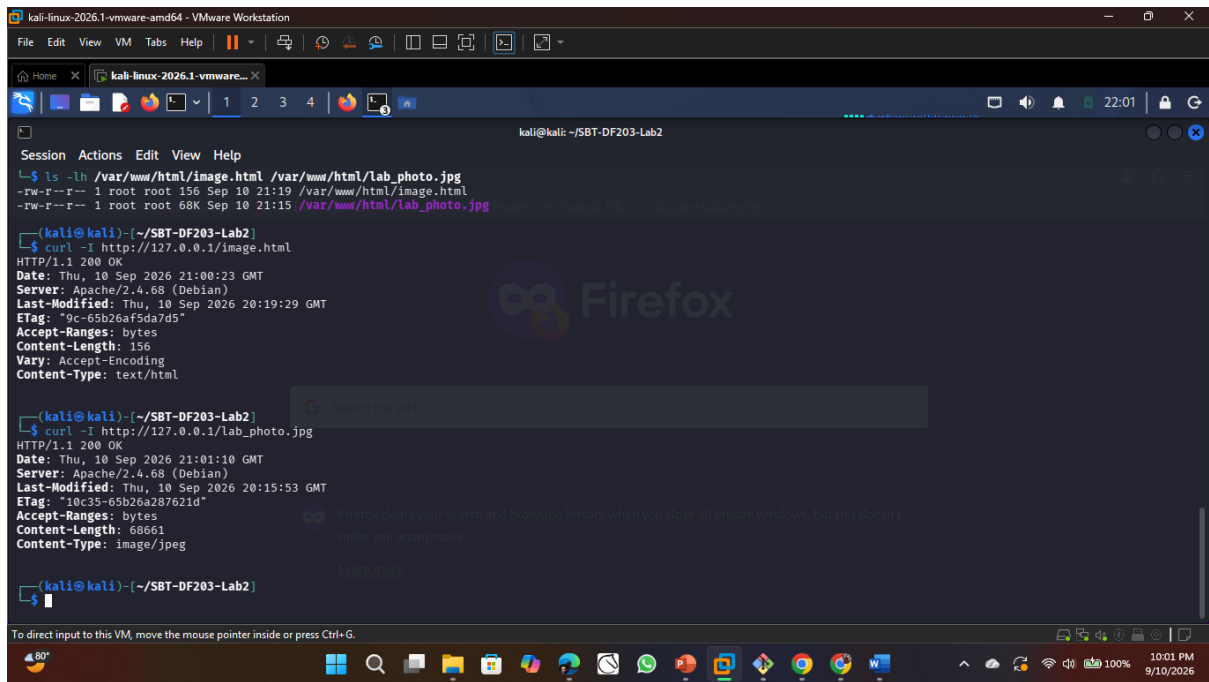
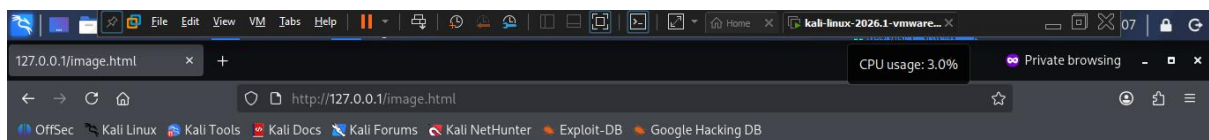


Fig 8: Checking both the html file and the image file size



SBT-DF203 Image Traffic

Analyst: OJE DOMINION MAYOWA



Fig 9: Verifying the Image and Text on the browser

Part B - Capture Browser-Generated HTTP Traffic

I opened a listening port on port 80 using tshark to be able to get generated http traffic from the browser


```
(kali㉿kali)-[~/SBT-DF203-Lab2]
$ tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
Capturing on 'Loopback: lo'
15 ^C

(kali㉿kali)-[~/SBT-DF203-Lab2]
$
```

Fig 10: Creating a listening port using tshark

```
(kali㉿kali)-[~/SBT-DF203-Lab2]
$ cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng

(kali㉿kali)-[~/SBT-DF203-Lab2]
$ sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
3f4c5db511e1c5d4b48e116815d5a1f150d9f40aacfc5160c6ab176c25eab46a evidence/image_traffic.pcapng
3f4c5db511e1c5d4b48e116815d5a1f150d9f40aacfc5160c6ab176c25eab46a working/image_traffic_working.pcapng

(kali㉿kali)-[~/SBT-DF203-Lab2]
$
```

Fig 11: Creating a copy of the evidence file and hashing both files to check for Integrity

Part C - Prove That Two Objects Were Requested

To prove that we requested two http objects. The two figures below indicates the GET request for image.html and the other for lab_photo.jpg.

```
(kali㉿kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host \
| tee reports/http_object_requests.tsv
4      2026-09-10T22:14:58.278377466+0100      0      127.0.0.1      49866      127.0.0.1      80      GET      /basic.html      127.0.0.1
8      2026-09-10T22:14:58.787861352+0100      0      127.0.0.1      49866      127.0.0.1      80      GET      /building.jpg      127.0.0.1

(kali㉿kali)-[~/SBT-DF203-Lab2]
$
```

Fig 12: GET request validation for Image.html

```
(kali㉿kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e http.content_length \
| tee reports/http_object_responses.tsv
6      2026-09-10T22:14:58.330384189+0100      0      200      text/html      174
14     2026-09-10T22:14:58.791625007+0100      0      200      image/jpeg      68661

(kali㉿kali)-[~/SBT-DF203-Lab2]
$
```

Fig 13: GET request for lab_photo.jpg

Part D - Analyze TCP Segmentation and Reassembly

Firstly, we had to check if the particular stream we were on and we discovered that both of our sources were on stream 0.


```
(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt

TCP Conversations
Filter:<No Filter>

127.0.0.1:49866 ↔ 127.0.0.1:80
Frames Bytes | Frames Bytes | Total | Relative | Duration |
7 69 kB | 8 1,461 bytes | 15 71 kB | 0.000000000 | 0.5154

(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng -T fields -e tcp.stream -e http.request.uri -Y 'http.request' | tee reports/streams_by_uri.txt
0 /basic.html
0 /building.jpg

(kali@kali)~/SBT-DF203-Lab2
$ tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields \
-e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
| tee reports/image_stream_segments.tsv
4 2026-09-10T22:14:58.278377466+0100 515 32 463 1 1
6 2026-09-10T22:14:58.330384189+0100 562 32 510 1 464
8 2026-09-10T22:14:58.787861352+0100 514 32 462 464 511
10 2026-09-10T22:14:58.790449593+0100 32820 32 32768 511 926
12 2026-09-10T22:14:58.791426932+0100 32820 32 32768 33279 926
14 2026-09-10T22:14:58.791625007+0100 3466 32 3414 66047 926

(kali@kali)~/SBT-DF203-Lab2
$
```

Fig 14: Listing the available frames or packets

From the Image we see that the GET request for /building.jpg was sent in frame 8, and the server's HTTP response header arrived in frame 14, showing 200 OK, Content-Type: image/jpeg, and Content-Length: 68661 bytes. Because this file is far larger than a single TCP segment can carry, Wireshark reassembled the full response from multiple underlying TCP segments (frames 10, 12, and 14) before displaying it as one complete HTTP object.

2. Contributing segments and their tcp.len values

Looking only at the image transfer (frames 10 onward, since frame 8 was just the GET request):

Frame	tcp.len (payload bytes)
10	32768
12	32768
14	3414

3. Sum of TCP payload lengths vs HTTP header + object data

Adding up the three contributing segments:

$32768 + 32768 + 3414 = 68950$ bytes

The HTTP Content-Length reported was **68661 bytes** (the JPEG object itself, not counting HTTP headers). The difference ($68950 - 68661 = 289$ bytes) accounts for the HTTP response headers (status line, Content-Type, Content-Length, Server, etc.) that were sent along with the object data in the same reassembled stream, since HTTP headers and body are transmitted together over TCP and only later separated logically by Wireshark's HTTP dissector.

4. Why segment sizes may differ from theoretical/slide values

The three segments carrying the image were not equal in size (32768, 32768, then a smaller 3414-byte remainder), and these numbers are noticeably larger than the standard Ethernet

MTU would normally allow per segment (~1460 bytes payload on a typical 1500-byte MTU link). This is mainly because:

- **Loopback interface has no real MTU limit** - capturing on `lo` bypasses the standard Ethernet MTU (1500 bytes) entirely, since traffic never touches a physical NIC, so the kernel is free to use a much larger local MTU (often 64KB or more).
- **TCP Segmentation Offload (TSO)** - modern network interfaces and operating systems can hand large chunks of data to the OS/NIC and let hardware/software split it into wire-sized segments only when needed, so what the capture shows as one large "segment" from the capturing tool's perspective doesn't necessarily reflect how it would look on a physical wire.
- **OS socket buffer sizes** - the amount of data the OS chooses to hand off at once depends on internal buffer sizes, which can vary between systems and configurations.
- **Capture location** - since this was captured on loopback (client and server on the same machine) rather than across a real physical network, the segmentation behavior differs from what would be seen on an actual Ethernet-based two-VM network, where the standard 1500-byte MTU would force much smaller, more numerous segments.

Part E - Export the Embedded Image

Using Wireshark we access the traffic image evidence in the working folder

```
(kali@kali)~/SBT-DF203-Lab2
$ wireshark working/image_traffic_working.pcapng &
[1] 1447912

(kali@kali)~/SBT-DF203-Lab2
$ ** (wireshark:1447912) 23:00:51.322454 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
```

Fig 15: Accessing the evidence traffic using Tshark

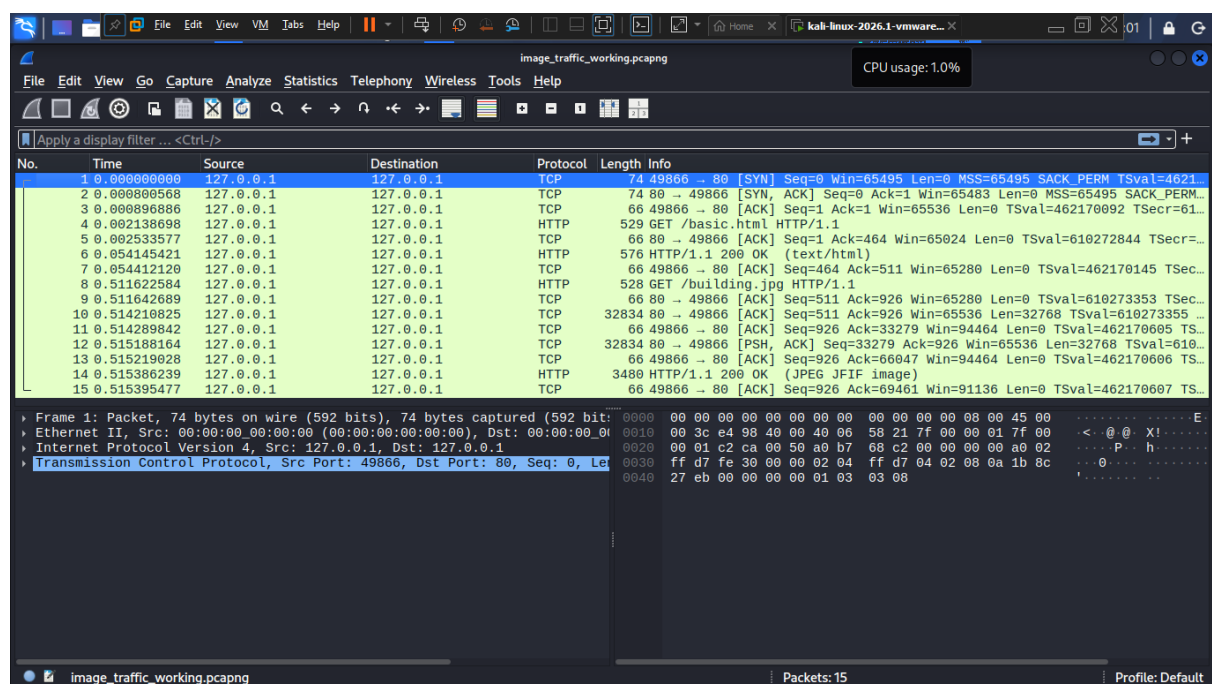


Fig 16: Wireshark terminal accessed successfully

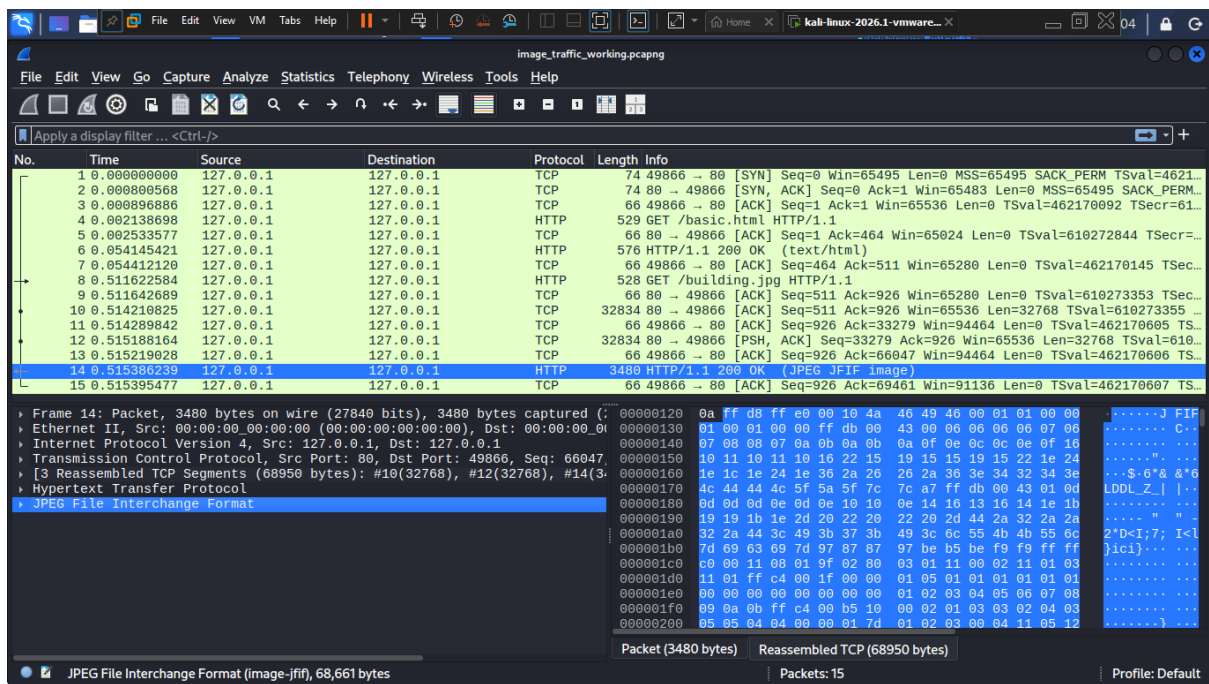


Fig 17: Checking the status of the Image on Packet 14

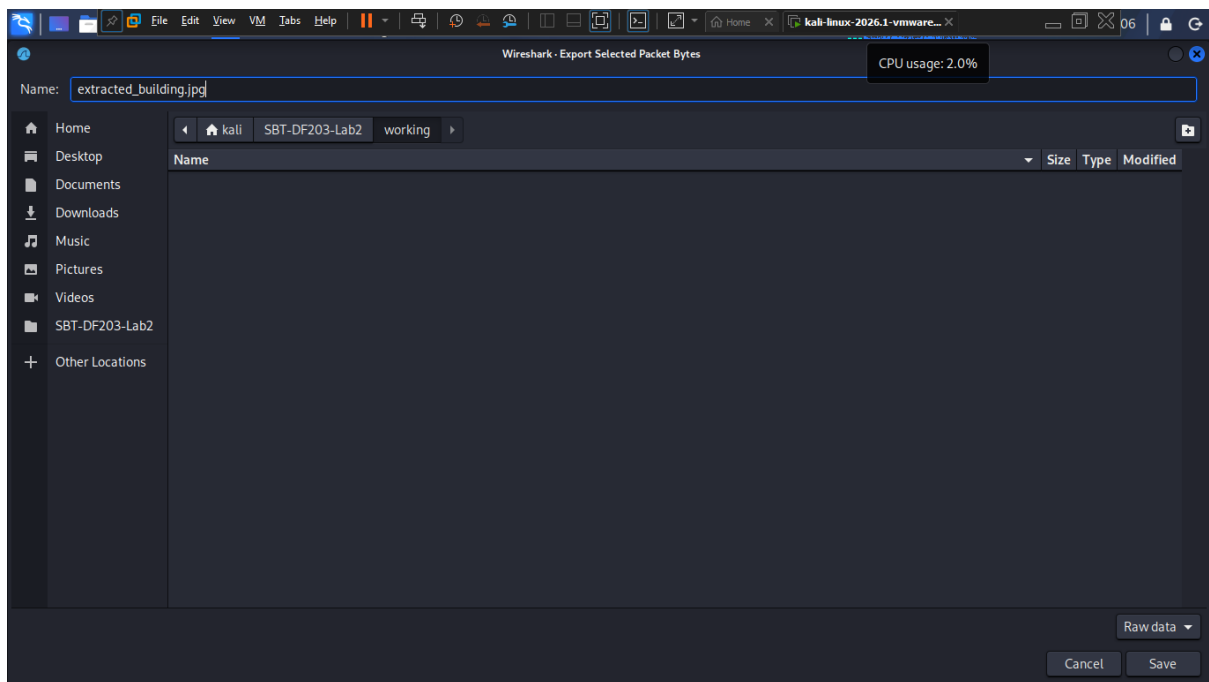


Fig 18: Exporting the Packet image in wireshark

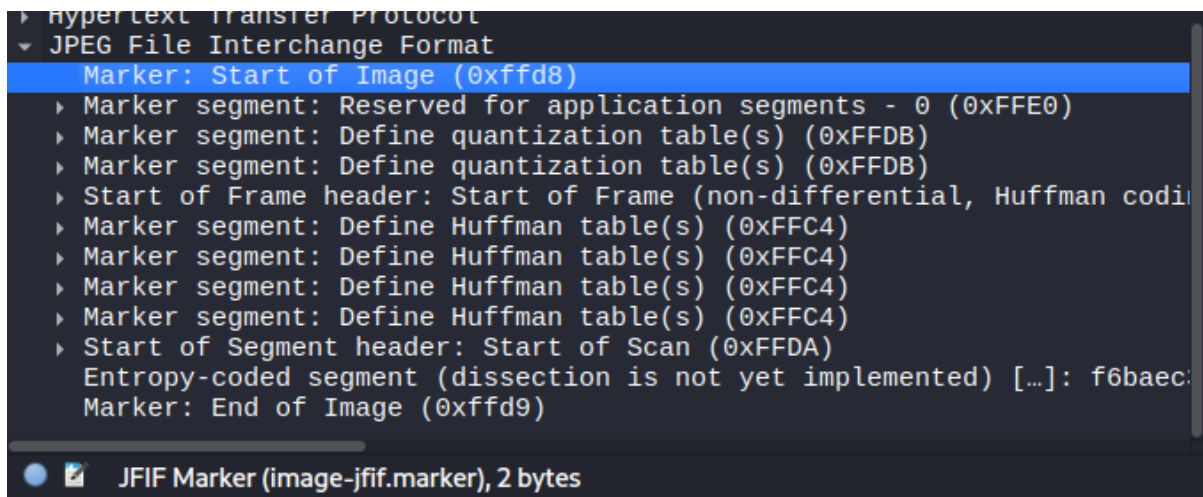


Fig 19: Locating the start and the end of the Image file

Using tshark as an Alternative

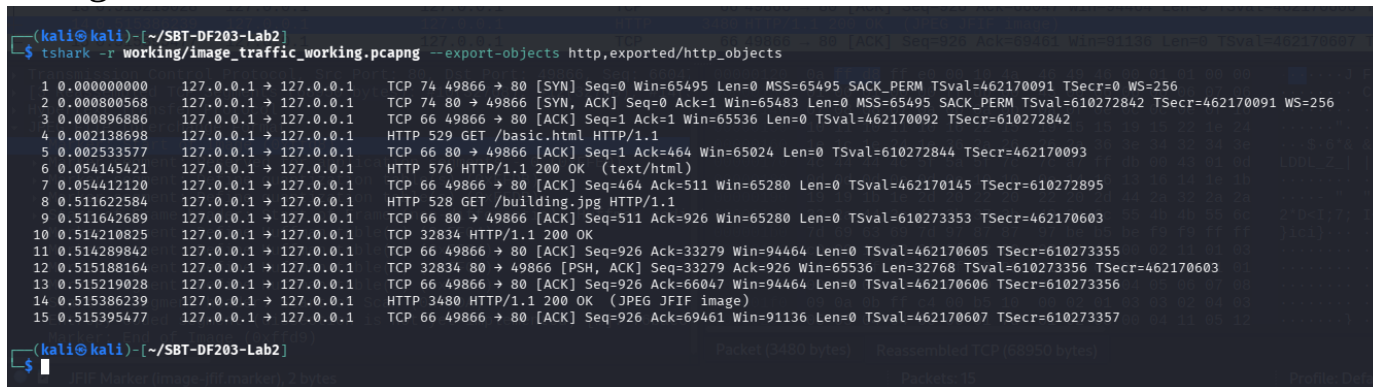


Fig 20: Using tshark to get the packets running and exporting it

Using the find and the file commands to get the number of files in the particular folder and the file type, file size, the dimensions and other necessary details of each files

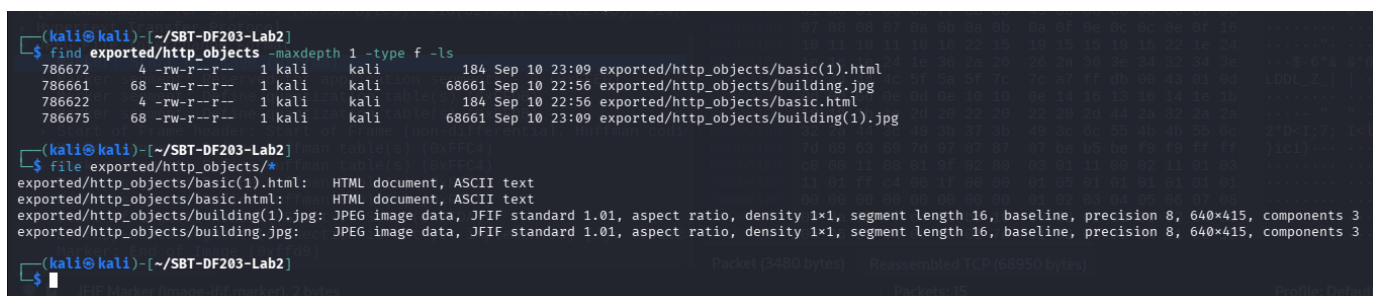
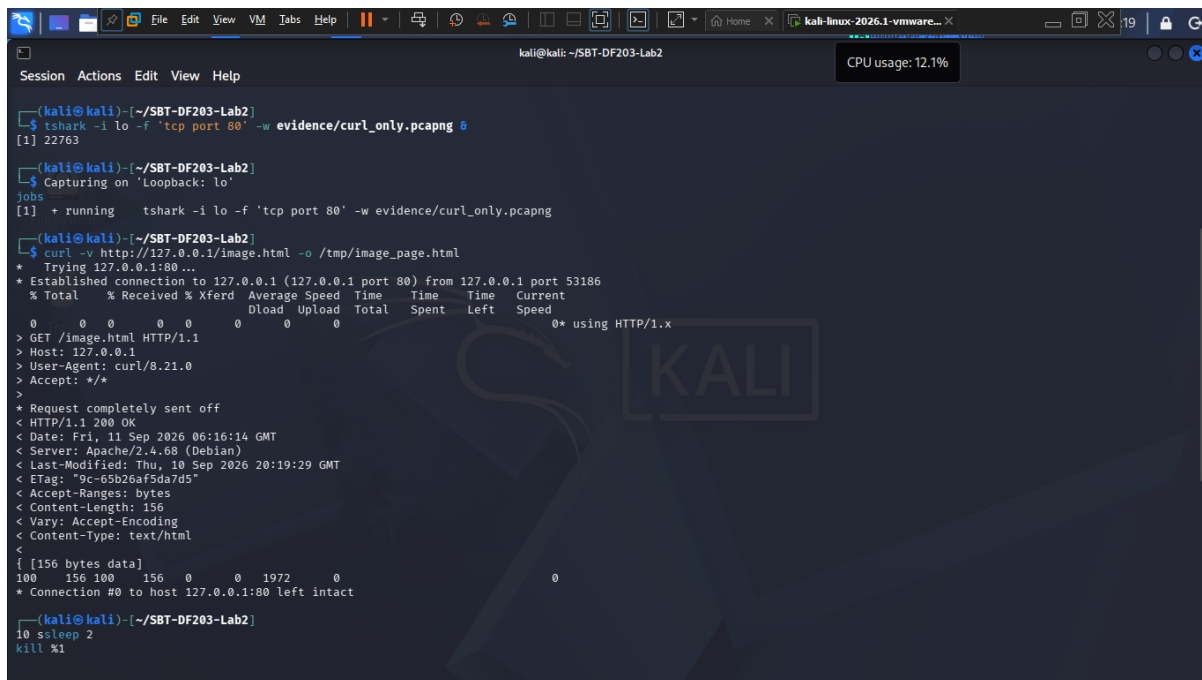


Fig 21: Displaying Exported Files

Part F - Compare curl and Browser Behavior

The first step was to allow the port 80 to listen and output it into the **curl_only.pcapng** file, then we checked if it actually running using the **jobs** command and then we ran curl of the **image.html** to allow it to be read by port 80 and captured.



```
(kali@kali)-[~/SBT-DF203-Lab2]
└─$ tshark -i lo -f 'tcp port 80' -w evidence/curl_only.pcapng
[1] 22763

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ jobs
[1] + running  tshark -i lo -f 'tcp port 80' -w evidence/curl_only.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 53186
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
0   0    0    0     0      0     0      0      0  0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 11 Sep 2026 06:16:14 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 20:19:29 GMT
< ETag: "9c-65b26af5da7d5"
< Accept-Ranges: bytes
< Content-Length: 156
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [156 bytes data]
100  156 100    0     0      0     0      0      0  0
* Connection #0 to host 127.0.0.1:80 left intact

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ sleep 2
└─$ kill %1
```

Fig 22: Listening and capturing using tshark and curl



```
(kali@kali)-[~/SBT-DF203-Lab2]
└─$ [1] + done  tshark -i lo -f 'tcp port 80' -w evidence/curl_only.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ ls -lh evidence/curl_only.pcapng
-rw-r--r-- 1 kali kali 1.9K Sep 11 07:17 evidence/curl_only.pcapng

(kali@kali)-[~/SBT-DF203-Lab2]
└─$ tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee reports/curl_requested_objects.txt
/image.html

(kali@kali)-[~/SBT-DF203-Lab2]
└─$
```

Fig 23: Checking the evidence file and if the .html file was captured

When I ran **curl -v http://127.0.0.1/image.html**, curl only sent one HTTP request, for **image.html** itself. Even though that page contains an **** tag, curl never issued a second request to fetch the image. The capture confirms this: **reports/curl_requested_objects.txt** shows only **/image.html**, with no request for **/building.jpg** anywhere in the traffic.

The main difference or why it didn't see any other thing apart from what it was asked to get is that this happens because curl is a raw HTTP client, not a browser. It has no HTML parser or rendering engine, so it simply fetches whatever URL it's given and stops. It has no way of knowing the page references another file, because that logic requires actually interpreting the HTML content, something curl was never built to do. WHILE A real browser behaves differently after downloading the HTML,

it parses the content, finds the ** tag**, and automatically sends a second GET request to retrieve that image so it can render the complete page visually.

Required Forensic Findings

Finding	Student Observation
Number of HTTP requests	2 in the browser-generated capture (GET /image.html, and GET /building.jpg on the stream 0)
HTML request URI and timestamp	/image.html :frame time 2026-09-10T22:14:58.278377466+0100 (frame 4, request)
Image request URI and timestamp	/building.jpg: frame time 2026-09-10T22:14:58.787861352+0100 (frame 8, request)
HTTP response codes	Both responses returned 200 OK
Image content type	Image/jpeg
Reported Content-Length	68661 bytes
Number of reassembled TCP segments	3 segments (frames 10, 12, 14)
Sum of TCP payload lengths	$32768 + 32768 + 3414 = 68950$ bytes
Original image SHA-256	a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb
Extracted image SHA-256	a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb
Hash comparison conclusion	Hashes matched exactly, confirming byte-for-byte recovery of the original image from the packet capture.
curl/browser difference	curl only requested the HTML file and did not automatically fetch the embedded image, since curl has no HTML parsing/rendering capability; a browser however parses the HTML and issues additional requests for referenced resources like images

This lab demonstrated how HTTP objects are transmitted, segmented, and reassembled over TCP, and how packet-level evidence can be used to recover files with full forensic integrity. By capturing browser-generated traffic, I observed how a single HTTP response can span multiple TCP segments and confirmed that the reassembled data matched the reported Content-Length. Extracting the embedded image directly from the packet capture and matching its SHA-256 hash against the original file proved that network captures can serve as

a reliable, verifiable source of digital evidence, byte-for-byte identical to the original, regardless of filename differences. Comparing curl and browser behavior further reinforced a key forensic principle: the tool used to generate traffic directly shapes what evidence appears in a capture, since automated clients like curl only retrieve what is explicitly requested, while browsers actively parse and fetch additional resources. Overall, this lab strengthened my understanding of how to capture, preserve, and validate network evidence using sound chain-of-custody practices.